DemoStream: Batched Task-and-Motion-Plan Optimization for Automated Learning from Demonstration

Anonymous Authors

***Abstract*— Learning from demonstration is an effective strategy for endowing robots with manipulation capabilities but requires training on datasets comprised of massive amounts of demonstrations. Existing demonstration collection approaches either require constant human involvement, for example, via teleoperation or are specialized for a single task. Task and Motion Planning (TAMP) systems are able to autonomously solve and therefore generate demonstrations for general-purpose tasks; however, learning from their demonstrations has proven difficult due to multi-modality in their solutions. In this work, we build on recent work leveraging GPU acceleration for TAMP and propose DemoStream, a TAMP system that optimizes demonstration quality in order to improve behavior cloning success rates. First, we introduce batched TAMP sampling operations along with a new TAMP algorithm that correspondingly performs batched constraint satisfaction in order to explore many solutions in parallel. We conduct simulation experiments ablating different action spaces, cost functions, and batch sizes to analyze the effect of these choices on downstream policy success rates. Finally, we show that we can leverage the safety guarantees of TAMP to directly collect real-world demonstrations at an average rate of 66 demos per hour. See our website for policy rollout videos: demostream.github.io.**

I. INTRODUCTION

Foundation models such as Vision-Language-Action Models (VLAs) [1] show incredible promise for robot control but require vast amounts of robot data, which is not easily obtainable. Teleoperation is a simple strategy for collecting robot data in the form of demonstrations but incurs a large human cost per task, making it difficult to scale. To reduce the amount of human effort, recent work developed semi-automated demonstration generation systems that adapt a handful of human-teleoperated demonstrations to new initial states [2]. However, these approaches still require annotated human demonstrations per task, preventing them from scaling autonomously to new tasks, and are typically deployed only in simulation because they produce unsafe behaviors.

Task and Motion Planning (TAMP) [3] systems are able to autonomously solve new tasks using the same planning model. Prior work has used TAMP to generate demonstrations for imitation learning [4], [5], but due to the stochastic nature of prior TAMP algorithms, the resulting demonstrations have been difficult to learn from due to their suboptimality and multi-modality. Recent work on TAMP leverages hardware acceleration, for example, on Graphics Processing Units (GPUs), to more efficiently solve problems [6], [7], [8]. We directly investigate the impact of GPU acceleration not only on producing demonstrations quickly but also on optimizing them so that they are easier to learn from.

Our approach, DemoStream, first makes novel TAMP contributions in the form of *batched streams*, conditional samplers that generate constraint-satisfying outputs for a batch of input values, and *batched computation graph* optimization algorithms that compose multiple batched streams and cost functions in order to explore thousands of TAMP solutions in parallel. These representation and algorithmic innovations give us the flexibility to explore multiple TAMP cost functions and analyze their effect on demonstration success rate, demonstration length, and downstream policy success rate. Notably, we find that without solution optimization, TAMP systems still have high demonstration generation success rates but train policies with success rates of zero (Table I). Additionally, the choice of cost function matters; we find that it is easier to learn from demonstrations that minimize task-space rather than joint-space costs, both during TAMP and motion planning, likely because these costs minimize the motion of wrist-mounted cameras, shrinking the distribution of encountered observations. DemoStream is able to train successful visuomotor policies on a variety of tasks including multi-robot (Figure 5), long-horizon (Figure 6), agentic open-world (Figure 7), and real-world (Figure 1) tasks.

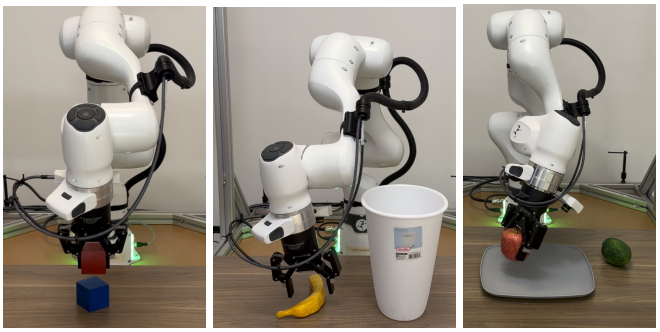


Fig. 1: **Real-World Policy Evaluation.** DemoStream collected 75 real-world demonstrations and trained visuomotor policies with a success rate of 67%. *Left*: stack the red block on the blue block (Stack-Two). *Center*: drop the banana into the trash bin (Banana-Bin). *Right*: pack the apple and lime on the plate (Apple-Lime).

The contributions of this paper are:

- New TAMP batched samplers and computation graph optimizers for GPU-accelerated plan optimization;
- Automated demonstration generation via agentic, batched TAMP for end-to-end imitation learning;
- Analysis on the effect of action space, cost function, and batch size on policy learning success rates;
- Simulation experiments demonstrating long-horizon, multi-robot, and open-world DemoStream capabilities;
- Safe automated real-world demonstration collection.

II. RELATED WORK

DemoStream builds on work at the intersection of demonstration generation for imitation learning and TAMP.

a) Demonstration Generation: We seek to acquire demonstration datasets of observation-action training examples to train visuomotor policies using Behavioral Cloning (BC) [9]. Rather than collecting these demonstrations through teleoperation [10], one strategy to reduce the amount of human effort involved is to teleoperate a handful of annotated demonstrations per task, adapt those demonstrations to new initial states, execute the adapted demonstrations, and retain the successful ones [2]. Non-trivial variations in the initial state induce large changes in the task-relevant workspace and safe configuration space, hampering the ability to successfully adapt demonstrations. Several approaches interleave skill-specific demonstration adaptation with motion planning to be robust to these variations [11], [12], [13], [14]. Still, these methods are *task-specific* and require new human demonstrations whenever the sequence of actions or involved objects change. In contrast, DemoStream is able to produce demonstrations for unseen tasks involving unseen objects without any human demonstrations.

b) Task and Motion Planning: Task and Motion Planning (TAMP) [3] jointly plans discrete & continuous robot actions, for example, which object it should pick and how it should safely execute the pick. Several TAMP algorithms [6], [8], [7] leverage GPU acceleration of some kind in order to more efficiently search over continuous action parameters. Of these, ScheduleStream [8] is closest to our approach; however, its use of GPU acceleration focuses on efficient multi-arm trajectory collision checking. In contrast, we introduce the notion of a batched conditional sampler, where a single operation generates many independent values at a time.

Two approaches [4], [5] investigated generating demonstrations fully autonomously using TAMP. Both used existing CPU-only TAMP algorithms to generate demonstrations [15], [16] without direct attention to optimizing demonstrations for learnability. In this work, we find that successfully solving tasks alone is not enough for successful imitation learning, one must also produce high-quality demonstrations. Moreover, these approaches were never shown to generate real-world demonstrations in a scalable manner.

Human-In-The-Loop (HITL) TAMP [17] interleaves TAMP with conventional teleoperation where TAMP delegates control to a human teleoperator during contact-rich segments of a task, such as insertion actions. Partial autonomy enables a single human to simultaneously teleoperate multiple robot arms using a queueing system; however, a human is still present on every episode.

III. BATCHED TAMP

Our goal is to develop a TAMP system that can autonomously generate high-quality demonstrations for imitation learning. To that end, we first provide background on the TAMP framework build off of (Section III-A). Then, we introduce *batched streams*, a new general-purpose TAMP representational capability that will allow us more efficiently optimize demonstration quality (Section III-B). In order to maximize the utility of batched streams, we present a new TAMP algorithm that composes batched streams in *batched computation graphs* (Section III-C).

A. TAMP Preliminaries

We build our approach on top of ScheduleStream [8], a domain-independent language for planning & scheduling with continuous sampling operations. ScheduleStream extends Planning Domain Definition Language (PDDL) [18] by offering support for Python sampling operations that generate continuous values, formalized as *streams*,

To convey our approach, we will use the single-robot [1] two-object packing task in Figure 6 (*left*) as a running example. In this task, the goal is to transfer both the `red` and `blue` blocks, which initially start on the `table`, to the `plate` in a manner that is efficient and avoids collisions. In this domain, the *constants*, *i.e.*, primitive values, are the discrete names of `objects = [red,blue,plate,table]`, continuous robot configurations $\mathtt{q} \in \mathbf{R}^d$, and continuous object placement and grasp poses $\mathtt{p}, \mathtt{g} \in \mathrm{SE}(3)$. Discrete values are strings and continuous values are PyTorch tensors.

Relationships among constants are represented using *functions*. A function **`Function`** represents a relationship between a tuple of inputs, modeled as *parameters* `"?x1 ... ?xN"`, and an output. A predicate **`Predicate`** is function where the output is either **`True`** or **`False`**. Predicates and functions specify a domain *condition*, denoted by *`cond`*, that conveys typing information about their parameters. In our running example, the following *predicates* declare constant typing information about their input parameters.

```
Conf = Predicate("?q"); Object = Predicate("?obj")
Placement = Predicate("?obj ?p")
Grasp = Predicate("?obj ?g")
```

Additionally, there are *fluent* functions that model changing state-variable values; `At` is the current robot configuration, `Holding` is what object the robot is holding (or **`None`**), `Attached` gives the parent frame of object `"?obj"` in the kinematic tree, and `Pose` is $\mathrm{SE}(3)$ pose of object `"?obj"` with respect to its parent frame.

```
At = Function(); Holding = Function()
Attached = Function("?obj", cond=[Object("?obj")])
Pose = Function("?obj", cond=[Object("?obj")])
```

States are *assignments* (denoted by `<=`) of output values to all function instances. Any undeclared function and predicate values are implicitly **`None`** and **`False`** respectively. The initial state in our example is:

```
initial_state = [At()<=q0, Holding()<=None,
  Pose(red)<=p_r, Pose(blue)<=p_b, Pose(plate)<=p_p,
  Attached(red)<=table, Attached(blue)<=table, ...],
```

where `q0` is the initial robot configuration and `p_r`, `p_b`, `p_p` are the initial poses of the `red`, `blue`, and `plate` objects. The goal conditions (denoted by `==`) are:

```
[Attached(red)==plate, Attached(blue)==plate].
```

[1] DemoStream also supports multiple robots (see the Multi-Arm task in Figure 5), going beyond prior TAMP demonstration generators [4], [5].

Actions describe legal decisions the system can take to change the current state. An action **Action** has a tuple of input *parameters* *params*, a list of equality *conditions* *cond*, a list of assignment *effects* *eff*, and a list of additive *costs* *cost*. An action can be applied to a state if all the conditions *cond* are satisfied by the state. After application, the assignments in the state are updated to have the values listed in the effects *eff* and the costs in *cost* are incurred.

In our example, we specify three actions [move, pick, place] although additional ones can readily be added. The move action has parameters for the current configuration "?q1", trajectory "?t", and new configuration "?q2".

```
move = Action(params="?q1 ?t ?q2",
  cond=[Motion("?q1 ?t ?q2"), At()=="?q1", ...],
  eff=[At()<="?q2"],
  cost=[Distance("?q1, ?q2")])
```

The pick action has parameters for the current robot configuration "?q1", object to be picked "?obj", current placement pose "?p", and new grasp pose "?g".

```
pick = Action(params="?q ?obj ?g ?p",
  cond=[Kin("?q ?obj ?g ?p"), Holding()==None,
        Pose("?obj")=="?p", At()=="?q"],
  eff=[Holding()<="?obj", Pose("?obj")<="?g",
       Attached("?obj")<=None])
```

The place action has parameters for the current robot configuration "?q1", object to be placed "?obj", current grasp pose "?g", new placement pose "?p", object to be placed upon "?o2", and the current pose of "?o2".

```
place = Action(params="?q ?obj ?g ?p ?o2 ?p2",
  cond=[Kin("?q ?obj ?g ?p"), Holding()=="?obj",
 Support("?obj ?p ?o2 ?p2"), Pose("?o2")=="?p2",
 Pose("?obj")=="?g", At()=="?q"] +
 [~Collision("?obj", Pose("?obj"), o, Pose(o))
  for o in objects],
  eff=[Holding()<=None, Pose("?obj")<="?p",
       Attached("?obj")<="?o2"])
```

B. Batch Streams

So far, we've modeled domains purely declaratively; however, we require a procedural component to generate values that serve as valid parameters *params* for actions [move, pick, place]. Additionally, these values must satisfy the following static predicates, which are present in function conditions and action preconditions *cond*. Grasp defines stable grasps "?g", Support defines placement poses "?p" supported by another pose "?p2", Kin models the kinematic constraint that the end-effector pose at configuration "?q" times grasp pose "?g" gives placement pose "?p", and Motion asserts configurations "?q1", "?q2" are the start and end of trajectory "?t".

```
Grasp = Predicate("?obj ?g", cond=[Object("?obj")])
Support = Predicate("?obj ?p ?o2 ?p2", cond=[
  Placement("?obj ?p"), Placement("?o2 ?p2")])
Kin = Predicate("?q ?obj ?g ?p", cond=[
  Conf("?q"), Grasp("?obj ?g"), Placement("?obj ?p")])
Motion = Predicate("?q1 ?t ?q2", cond=[
  Conf("?q1"), Conf("?q2")])
```

PDDLStream [16] proposed generating predicate-satisfying constants by modeling *streams*, conditional generators that take in a tuple of input constants and yield tuples of output constants. A stream, denoted as **Stream**, specifies a predicate, a subset of the predicate's parameters as inputs *cond* (the remaining parameters are outputs), and a Python function that returns a generator *gen*. For example, a grasps stream takes object "?obj" as input and yields grasp pose tensors for output parameter "?g".

```
def grasp_gen(obj: str) \
    -> Iterator[Tuple[Tensor]]: ...
grasps = Stream(Grasp, inps="?obj", gen=grasp_gen)
```

While it is possible to leverage GPU acceleration within grasp_gen to produce many outputs, fundamentally streams consume only a single input at a time. This becomes limiting when we consider an Inverse Kinematics (IK) stream ik_qs responsible for satisfying Kin. We would like to leverage advances in batched IK [19] to solve many IK subproblems simultaneously, for different grasp and placement combinations, but to do so, we must augment the stream contract. To that end, we introduce *batch streams*. A batch stream, **BatchStream**, specifies a batched Python function *fn*, which takes in a list of input tuples and outputs a list of lists of output tuples per input tuple, along with a maximum batch size *batch*. For example, the ik_qs stream in Figure 2 takes $N = 128$ grasp and placement pairs as input and solves for configurations that satisfy them.

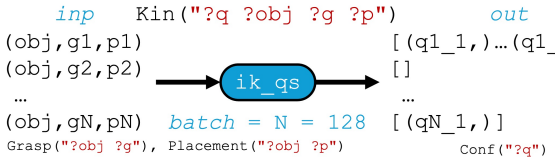


Fig. 2: **Batch IK Stream.** The ik_qs stream takes a batch of $N = 128$ inverse kinematics subproblems as inputs and returns a list of configurations "?q" per subproblem, if any can be found.

In our domain, we batch the placement generation, IK, and motion streams placements, ik_qs, and motions because they all take continuous values as inputs.

```
def support_fn(inp: List[Tuple[str,str,Tensor]]) \
    -> List[List[Tuple[Tensor]]]: ...
def ik_fn(inp: List[Tuple[str,Tensor,Tensor]]) \
    -> List[List[Tuple[Tensor]]]: ...
def motion_fn(inp: List[Tuple[Tensor,Tensor]]) \
    -> List[List[Tuple[Tensor]]]: ...
placements = BatchStream(Support,
  inps="?obj ?o2 ?p2", fn=support_fn, batch=inf)
ik_qs = BatchStream(Kin,
  inps="?obj ?g ?p", fn=ik_fn, batch=256)
motions = BatchStream(Motion,
  inps="?q1 ?q2", fn=motion_fn, batch=1)
```

We also batch functions that compute configuration distances (Distance) and placement collisions (Collision).

```
def distance(inp: List[Tuple[Tensor,Tensor]]) \
    -> List[float]:
def collision(inp: List[Tuple[str,Tensor, \
    str,Tensor]]) -> List[bool]:
Distance = Function("?q1 ?q2",
  fn=distance, batch=inf)
Collision = Predicate("?obj1 ?p1 ?o2 ?p2",
  test=collision, batch=inf)
```

C. Batch Planning

To effectively plan with the batch stream in Section V-C, we introduce a new planning algorithm BATCH-STREAM. It

builds off of the LAZY-STREAM algorithm [8] in that it plans lazily prior to calling streams to avoid unnecessary expensive operations. Critically, it differs in how it calls streams once it has identified a plan in BATCH-OPTIMIZATION.

Algorithm 1 Batch Stream Planning

procedure BATCH-STREAM($\mathcal{I}, \mathcal{G}, \mathcal{A}, \mathcal{S}; c_{\max}$)
  $S \leftarrow$ LAZY-STREAMS($\mathcal{I}, \mathcal{S}$) ▷ Create lazy versions of the streams
  while True do
    $I \leftarrow$ APPLY-STREAMS($\mathcal{I}, S$) ▷ Propagate lazy stream outputs
    $\pi \leftarrow$ SEARCH($I, \mathcal{G}, \mathcal{A}$) ▷ Solve a search subproblem
    if $\pi =$ **None then**
      return None ▷ Return failure
    $\psi \leftarrow$ RETRACE-STREAMS($\mathcal{I}, S, \pi, \mathcal{G}$) ▷ Lazy plan parameters
    $F \leftarrow$ EXTRACT-COSTS(π) ▷ Extract action costs
    $b_* \leftarrow$ BATCH-OPTIMIZATION($\psi, F; c_{\max}$) ▷ Bind parameters
    if $b_* \neq$ **None then**
      return SUBSTITUTE(π, b_*) ▷ Return a solution
    $S \leftarrow$ UPDATE-STREAMS(S, ψ) ▷ Remove attempted streams

Algorithm 1 displays the pseudocode for BATCH-STREAM. It takes as input initial state $\mathcal{I}$, goal conditions $\mathcal{G}$, actions $\mathcal{A}$, and batched streams $\mathcal{S}$ as well as a maximum cost $c_{\max} \in [0, \infty)$. First, it creates a set of *lazy* batched streams S, with placeholder output values [8]. On each iteration, it constructs an augmented initial state $I \supseteq \mathcal{I}$ using the lazy stream outputs. Then, it solves a finite search subproblem. If a plan π is found, it identifies the lazy action parameters along the plan and retraces the streams associated with them, producing a *stream plan* $\psi \subseteq S$. A stream plan can be thought of as a serialized *computation graph* for binding action parameters. It also extracts the function costs F from the plan. Then, it calls BATCH-OPTIMIZATION, the key subroutine that attempts to bind values to these lazy action parameters using batched stream calls (Algorithm 2). The lowest-cost binding b_* is used to bind the lazy action parameters on π, and the resulting plan is returned. If no satisfying binding is found, the set of lazy streams S is updated to remove the streams attempted in stream plan ψ.

Algorithm 2 Batch Stream Optimization

procedure BATCH-OPTIMIZATION(ψ, F; $c_{\max}$)
  $B = [\{\}]$ ▷ Initialize list of bindings with an empty binding
  for $\langle s, ?x, ?y \rangle \in \psi$ **do** ▷ Stream s, input $?x$, output $?y$
    $B' \leftarrow [\,]$ ▷ List of new bindings
    $X \leftarrow$ [SUBSTITUTE($?x, b$) **for** $b \in B$] ▷ Batch inputs
    for $\langle x', y' \rangle \in$ **zip**$(X, s.fn(X))$ **do** ▷ Batch stream call
      for $b \in B$ **do**
        if SUBSTITUTE($?x, b$) $= x'$ **then** ▷ x' in binding b
          $B' \leftarrow B' + [b \cup \{?y : y'\}]$ ▷ Augment binding
    $B \leftarrow B'$ ▷ Sort by increasing cost
  $C \leftarrow [0.] * |B|$ ▷ Initialize zero cost per binding
  for $\langle f, ?z \rangle \in F$ **do** ▷ Function f, input $?z$
    $Z \leftarrow$ [SUBSTITUTE($?z, b$) **for** $b \in B$] ▷ Batch inputs
    $C \leftarrow C + f.fn(Z)$ ▷ Batch function call
  if $|B| = 0$ **or min**$(C) > c_{\max}$ **then**
    return None ▷ No satisfying bindings
  return B[**argmin**(C)] ▷ Return the best binding

Algorithm 2 displays the pseudocode for BATCH-OPTIMIZATION, a algorithm for batched computation graph optimization. Although we use it as a subroutine, it can be used for general-purpose constraint satisfaction to, for example, solve for collision-free simulation states. It takes in a stream plan ψ, a set of function costs F, and a maximum cost $c_{\max}$. It maintains a list of bindings B, mappings from lazy action parameters to values. It processes each stream instance $\langle s, ?x, ?y \rangle$ in the stream plan ψ, where $?x$ are input parameters and $?y$ are output parameters. On each iteration, it concatenates binding values for input parameters $?x$ into an input batch X. Then, it calls stream s, generating a batch of output values y' associated with input values $x' \in X$. A new list of bindings B' is created by joining the existing bindings B with the new parameter values y'.

After all streams in ψ have been called, BATCH-OPTIMIZATION computes the cost of each binding C in batch. In practice, this is interleaved with stream evaluation, where a function is evaluated as soon as its parameter values can be bound, so that the lowest-cost bindings are greedily prioritized in batch streams. BATCH-OPTIMIZATION fails if there are no bindings or if the minimum binding cost is greater than the input maximum cost $c_{\max}$. Otherwise, it returns the minimum cost binding in B. In our implementation, BATCH-OPTIMIZATION runs on the CPU but each stream and function call runs on the GPU; however, all these operations could be performed on the GPU [7] to maximize performance at the expense of flexibility.

As an example, suppose SEARCH in BATCH-STREAM returns the following plan π for placing `red` on the `plate`.

```
plan = [move(q0,"?t1","?q1"),
  pick("?q1",red,"?g",p_r),
  move("?q1","?t2","?q2"),
  place("?q2",red,"?g","?p",plate,p_p)]
```

RETRACE-STREAMS and EXTRACT-COSTS will produce the interleaved stream plan ψ and function costs F in Figure 3 corresponding to plan π. Suppose `grasps` and `placements` each generate 1,000 outputs per input. The bottleneck in the stream plan ψ is then `ik_qs`, which has a batch size of 128; however, through sorting by costs and filtering by constraints, the best 128 bindings are prioritized.

IV. Demonstration Generation

Although in Section III we make novel representation and algorithmic contributions to TAMP itself, these contributions are motivated by investigating the impact of TAMP demonstration quality on visuomotor policy learning. Here, we describe how DemoStream solves TAMP problems to produce demonstration datasets for imitation learning (Section IV-A), is implemented (Section IV-B), supports multiple cost functions (Section IV-C) and motion planners (Section IV-D), and integrates with coding agents (Section IV-E).

A. TAMP Demonstrations

We are interested in generating datasets of observation-action trajectories that successfully complete a task. Here, observations consist of robot proprioception, including the joint positions, gripper position, and end-effector pose, as well as wrist and one or more external camera RGB images. We consider three action spaces in particular to understand whether the best configuration of DemoStream depends on

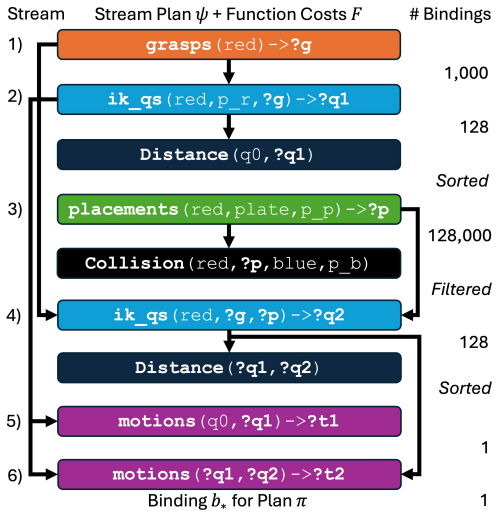


Fig. 3: **Batched Computation Graph.** The computation graph (stream plan ψ) corresponding to a plan π involving picking the red block and placing it on the plate. The right column lists the number of bindings $|B|$ after each batch stream call.

the action space (Section V-A). Specifically, we experiment with joint-space absolute (Joint-Abs) and relative (Joint-Rel) actions as well as task-space relative actions (IK-Rel), implemented using an Inverse Kinematics (IK) controller.

To produce a dataset, we first interpret the goal $\mathcal{G}$, for example, using a Large Language Model (LLM) coding agent [20]. Then, we sample an initial state, convert it to a planning state $\mathcal{I}$ and solve a TAMP problem for a plan π that achieves the goal $\mathcal{G}$. We execute plans open-loop, retain the resulting observations and actions as demonstrations, and save the demonstrations that reach a goal state We use a fixed set of actions $\mathcal{A}$ and streams $\mathcal{S}$ across all our experiments in Section V, although additional capabilities can easily be authored by LLM coding agents. After we generate 1,000 successful demonstrations, we train discriminative [10] and generative [21] supervised models that distill the TAMP solver by mapping directly from observations to actions.

We take advantage of batched simulated environments [22] to try hundreds of TAMP solutions in parallel across different initial states. Batched simulation negates the simulation bottleneck, leaving TAMP as the bottleneck at several seconds per successful demonstration (Figure V-C). While we could correspondingly batch TAMP to solve multiple problems simultaneously by modeling an initial-state-generating action and stream, this comes at the expense of optimizing solution quality given a finite amount of GPU memory.

B. Stream Implementation

Like in prior work [19], we represent the collision volume of the robot as a union of spheres greedily optimized to cover its underlying mesh model [8]. We use cuRoboV2 [23] for batched multi-pose inverse kinematics, implementing `ik_fn`. We apply GPU acceleration beyond motion planning by also using it during grasp and placement generation. We implement `grasp_fn` by developing a vectorized antipodal grasp sampler that generates 10,000 antipodal points on an object, samples rolls about these axes, filters out grasps that never make contact with the object, and filters out grasps in collision with the object when fully open. We perform these collision checks efficiently by transforming just the spheres on the robot's end-effector. Not all grasps will result in successful execution (Figure 7); however, through trial-and-error execution, only successful grasps are retained for policy learning. We adapt a similar strategy to implement `placement_fn` by randomly generating 10,000 candidate placements of object `"?obj"` above `"?o2"` using their bounding boxes, filtering out placements that do not make contact with `"?o2"` in the $-z$ direction, and filtering out placements that are in collision with `"?o2"`. We use asymmetric collision checking between movable objects to implement `collision` as the conjunction of collisions between spheres of object `"?obj1"` and the mesh of object `"?o2"` and vice versa [8].

C. Cost Functions

The choice of TAMP cost function to implement `distance`, along with how much computation is spent optimizing it, directly impacts downstream policy learning. Optimizing a cost function implicitly enforces regularity on the distribution of TAMP solutions and thus the distribution of demonstrations. We hypothesize that it is easier for policies to learn when the distribution of demonstrated actions per state is unimodal and concentrated, even for generative models like Diffusion Policy [21]. Concretely, the implementation of `distance` affects which grasps, placements, and configurations are selected within BATCH-OPTIMIZATION, serving as targets for motion planning.

In Section V-A, we perform an experiment ablating the following three cost functions and notice dramatic effects on policy learning: *Zero* cost, *Joint*-space distance, and end-effector *Task*-space distance. A zero cost function makes TAMP indifferent to the choice of solution, essentially sampling a feasible solution at random [5]. The joint-space distance is the most natural function because it correlates with the execution duration of the plan. The end-effector task-space distance computes a weighted sum of the translation and rotation of an end-effector link. Minimizing this cost function minimizes the motion of every link rigidly connected to the end-effector, notably wrist-mounted cameras.

D. Motion Planning

Building on our investigation of joint- versus task-space TAMP cost functions (Section IV-C), we also explored multiple ways to plan low-level motions, implementing `motion_fn`. We first consider an off-the-shelf motion planner: cuRoboV2's joint-space trajectory optimizer (cuRoboV2) [23]. To have full control over planning, we also developed a batched sampling-based motion planner applicable both in joint and task space. It takes as a hyperparameter a batched cost function modeling collision and motion edge costs. The planner has two phases: a global exploration phase that creates a Probabilistic Roadmap (PRM) [24] in batch and a local exploitation phase that performs shortcutting [25] in

batch. When planning in task space, we sample end-effector poses using batched forward kinematics on uniformly sampled configurations. We check collisions only with the end-effector collision spheres, and lazily map plans to joint space using iterative IK. We consider the following motion planners: joint-space PRM with joint-space costs (J-PRM), joint-space PRM with task-space costs (J-PRM:T), and task-space PRM with task-space costs (T-PRM).

E. Agentic Goals

We are interested in generating demonstrations for tasks with novel open-world goals, for example, those described by natural language. For example, consider the Rubiks-Near and Rubiks-Left RoboLab [26] tasks in Figure 7, where the goals are for the Rubik's cube to be near and left of the bowl. With the rise of LLM coding agents, we adopt the strategy of OWL-TAMP [20] by parameterizing object-object goal predicates with a natural language prompt parameter. Critically, our approach differs in that the agent must implement a batched version of the test that is compatible with efficient planning. LLM coding agents can implement batched tests upfront using offline skills when the set of tasks is known or lazily online to respond to unseen goals. For simulation tasks with programmatic success tests, LLM coding agents can even read the implementation of the test and adapt them to support GPU-accelerated batched planning.

```
goal = [AgenticGoal("Rubik's cube left of the bowl",
        rubiks, Pose(rubiks), bowl, Pose(bowl))]
```

V. EXPERIMENTS

We conducted experiments to understand how the following factors affect policy Success Rates (SR): 1) action space and cost function (Section V-A), 2) demonstration generator (Section V-B), 3) batch size (Section V-C), 4) task variation (Section V-D), 5) complicated objects (Section V-E), 6) real-world constraints (Section V-F).

Unless indicated otherwise, we used the IsaacLab [22] physics simulator, IK-Rel action space, collected 1,000 demonstrations using DemoStream, added $[-0.01, +0.01]$ uniform noise to actions, used an IK batch size of 128, trained visuomotor BC-RNN policies [10] or Diffusion Policies [21] for 600 and 2,000 epochs, performed 50 rollouts per policy, and reported the best-performing checkpoint from the latter half of training.

A. Action Space and Cost Function

Our first experiment studies the effect of TAMP (Section IV-C) and motion (Section IV-D) cost functions on policy learning performance in different action spaces (Section IV-A). Table I displays the results of the experiment. Minimizing task-space costs (Task) and using a task-space PRM (T-PRM) results in the highest success rates across action spaces. We also trained Diffusion Policy [21] on this data and found it had even higher success rates of 86% (Joint-Abs), 90% (Joint-Rel), and 86% (IK-Rel). The joint-space PRM with a task-space metric (J-PRM:T) outperforms cuRoboV2 and J-PRM, showing that even soft end-effector motion minimization is beneficial. The zero cost function (Zero), which is representative of prior work [5], fails to train policies with positive success rates in both the Joint-Abs and Joint-Rel action spaces. Additionally, joint-space costs with cuRoboV2 is representative of cuTAMP [6], which DemoStream also outperforms. We hypothesize that minimizing wrist-camera motion is critical for effective visuomotor policy learning and note that randomized planners can generate high-quality demonstrations via batched optimization.

TABLE I: **Cost Ablation.** BC-RNN policy success rates on the Stack-Three task across action spaces, cost functions, and motion planners. Minimizing task-space TAMP and motion costs maximizes policy success rate across action spaces.

Action Space	Cost	cuRoboV2	J-PRM	J-PRM:T	T-PRM
Joint-Abs	Zero	0%	0%	0%	0%
	Joint	22%	12%	32%	34%
	Task	2%	0%	0%	**76%**
Joint-Rel	Zero	0%	0%	0%	0%
	Joint	6%	10%	26%	38%
	Task	0%	0%	0%	**72%**
IK-Rel	Zero	46%	20%	34%	36%
	Joint	46%	56%	76%	42%
	Task	72%	70%	78%	**82%**

B. MimicGen Comparison

To understand how DemoStream compares to existing non-TAMP demonstration generation techniques, we compared DemoStream to IsaacLab [22]'s default implementation of MimicGen [2] using the reference demonstrations. Table II shows that although each MimicGen episode is faster than a DemoStream episode, the MimicGen success rate is substantially lower than that of DemoStream (35% vs. 92%) and as a result the time to collect a successful demo is higher (9s vs. 6s). Additionally, MimicGen both produced longer demonstrations because it doesn't optimize for the best feasible demonstration per problem and trained a worse-performing policy. DemoStream outperformed MimicGen all while not requiring any annotated human demonstrations.

TABLE II: **MimicGen vs. DemoStream**. DemoStream results in higher demonstration and policy Success Rates (SR) than MimicGen [2], without any task-specific annotated human demonstrations.

Method	Demo SR	Demo Length	Time/Demo	Policy SR
MimicGen [2]	35%	243 ± 16	9s	34%
DemoStream	**93%**	113 ± 6	**6s**	**82%**

C. Batch Size Scaling

We conducted a scaling experiment to assess how DemoStream performs as the IK stream batch size is scaled from 1 to 256 by factors of 4. Here, an IK batch size of 1 is representative of the performance of prior CPU-only TAMP methods that produce just feasible solutions [5]. Figure 4 shows that as the batch size increases, the demo success rate increases, the mean demonstration length decreases and its distribution narrows, and the policy success rate increases. In this experiment, the mean and standard deviation of

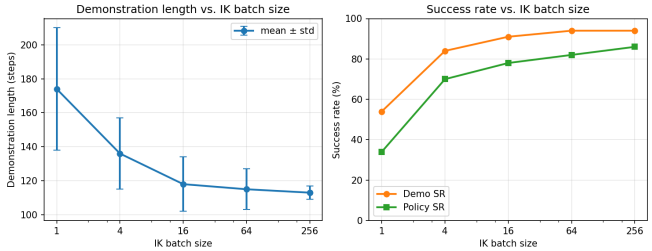


Fig. 4: **Scaling Batch Size.** The effect of scaling the IK batch size by multiples of 4. *Left*: the mean and standard deviation of the demonstration length. *Right*: the demonstration and policy Success Rate (SR). Large batch sizes decrease demonstration length and dispersion, resulting in higher policy success rates.

demonstration lengths correlate with policy success rate. To understand whether simply short demonstrations were easier to learn, we halved and quartered the DemoStream execution speed, resulting in longer demonstrations, but found that the policy success rate actually slightly increased to 90% and 92% respectively. Thus, mean demonstration length alone is not a good predictor of policy success rate, but Coefficient of Variation, the ratio of the standard deviation to the mean, correlates meaningfully with policy success rate.

D. TAMP Tasks

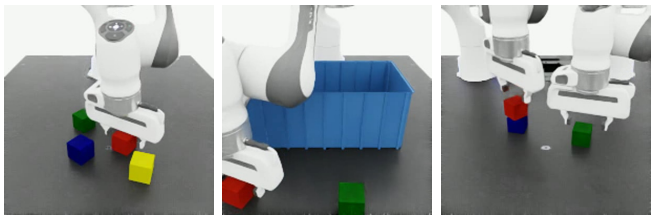


Fig. 5: **Stacking Tasks.** *Left*: stack four blocks (Stack-Four, long-horizon). *Center*: stack three blocks in the bin (Stack-Bin, obstacles). *Right*: stack three blocks using two arms (Multi-Arm).

We tested DemoStream on more difficult TAMP tasks [16] requiring long horizons (Stack-Four), collision-aware motion (Bin-Stack), collision-aware placement (Pack-Two, 100% Blocked), varying task plans (Pack-Either, 50% Blocked), and multiple robots (Multi-Arm). We defined four stacking tasks (Figure 5): stack two (Stack-Two) and four blocks (Stack-Four), stack three blocks in a bin (Bin-Stack), stack three blocks using two arms (Multi-Arm). We used absolute joint-space actions in Multi-Arm. We also considered the following four packing tasks (Figure 6): pack two blocks on the plate (Pack-Two), pack either block on the plate (Pack-Either), and pack the red block where the blue block starts on the plate with 50% (50% Blocked) and (100% Blocked) probability. Table III shows that DemoStream is able to train high-performing policies despite increased task difficulty.

E. RoboLab Tasks

To validate that DemoStream works on more complex objects and open-world goals (Rubiks-Near, Rubiks-Left), we experimented on several RoboLab [26] IsaacLab Arena tasks (Figure 7). The tasks are: place the Rubik's cube in

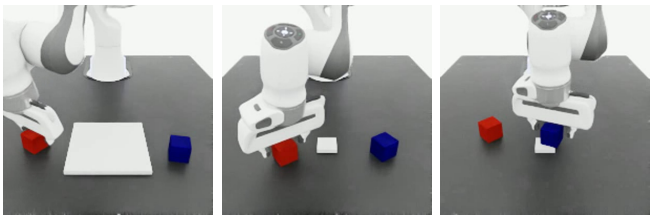


Fig. 6: **Packing Tasks.** *Left*: pack the two blocks on the plate (Pack-Two). *Center*: pack the red block on the plate (50% Blocked). *Right*: pack the red block on the obstructed plate (100% Blocked).

TABLE III: **Stacking and Packing Results.** The Demonstration Success Rate (Demo SR) and policy Success Rate (SR) across the stacking (Figure 5) and packing (Figure 6) tasks.

Task	Demo SR	BC-RNN SR	Diffusion SR
Stack-Two	100%	100%	90%
Stack-Four	72%	54%	64%
Bin-Stack	97%	82%	98%
Multi-Arm	54%	12%	50%
Pack-Two	93%	94%	96%
Pack-Either	94%	90%	82%
50% Blocked	99%	88%	86%
100% Blocked	99%	94%	90%



Fig. 7: **RoboLab Tasks.** *Left*: put the Rubik's cube and banana in the bowl (Rubiks-Banana). *Center*: place the mustard on the raisins (Mustard-Raisin). *Right*: drop the mustard in the bin (Mustard-Bin).

the bowl (Rubiks-Bowl), place the Rubik's cube near the bowl (Rubiks-Near), place the Rubik's cube left of the bowl (Rubiks-Left), place the banana in the bowl (Banana-Bowl), place the Rubik's cube and banana in the bowl (Rubiks-Banana), place the mustard bottle on the raisin box (Mustard-Raisin), and drop the mustard bottle in the bin (Mustard-Bin). Table IV shows that although DemoStream requires more attempts to stably grasp objects, it distills effective policies.

TABLE IV: **RoboLab Results.** Demonstration Success Rate and policy Success Rate (SR) on the RoboLab tasks (Figure 7).

Task	Demo SR	BC-RNN SR	Diffusion SR
Rubiks-Bowl	51%	86%	76%
Rubiks-Near	63%	96%	56%
Rubiks-Left	27%	48%	28%
Banana-Bowl	40%	78%	44%
Rubiks-Banana	3%	18%	22%
Mustard-Raisin	55%	88%	34%
Mustard-Bin	17%	86%	70%

F. Real World

We conducted a real-world demonstration-collection and policy-learning experiment on a DROID [27]-style Franka Panda to understand DemoStream's viability for direct real-world learning. The human input to the system is a

command-line natural language prompt describing the task (Section IV-E). Unlike in simulation, TAMP cannot directly read the geometry and pose of objects. To address this, we adopted the strategy of Curtis *et al.* [28] for open-world TAMP, where object geometries are generically estimated from a point cloud segmented by SAM3 [29].

TABLE V: **Real World Results.** The Demonstration Success Rate (Demo SR), demonstration length, time per demo, and policy Success Rate (SR) on the real-world tasks (Figure 1).

Real Task	Demo SR	Demo Length	Time/Demo	Policy SR
Stack-Two	72%	178 ± 13	48s	65%
Banana-Bin	83%	214 ± 19	49s	75%
Apple-Lime	79%	323 ± 28	69s	60%

We considered three tasks (Figure 1): stack the red block on the blue block (Stack-Two), drop the banana in the bin (Banana-Bin), and pack the apple and lime on the plate (Apple-Lime). DemoStream collected 75 demonstrations per task at a throughput of 66 demonstrations per hour. The pipeline was autonomous without intervention aside from human-performed randomized resets and success labeling; for certain tasks, TAMP could automate these low-effort operations as well. We trained a Joint-Abs Diffusion Policy [21] per task. The policy's observations were proprioception as well as 200x200 wrist and left-external camera images. We evaluated the final checkpoints over 20 task trials. Table V shows that DemoStream trains policies that are successful on average 67% of the time on purely real-world data. See our website for rollout videos: https://demostream.github.io. [2]

REFERENCES

[1] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang *et al.*, "Gr00t n1: An open foundation model for generalist humanoid robots," *arXiv preprint arXiv:2503.14734*, 2025.

[2] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, "Mimicgen: A data generation system for scalable robot learning using human demonstrations," in *7th Annual Conference on Robot Learning*, 2023.

[3] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, "Integrated Task and Motion Planning," *Annual Review of Control, Robotics, and Autonomous Systems*, 2021.

[4] M. J. McDonald and D. Hadfield-Menell, "Guided imitation of task and motion planning," in *Conference on Robot Learning*. PMLR, 2022, pp. 630–640.

[5] M. Dalal, A. Mandlekar, C. Garrett, A. Handa, R. Salakhutdinov, and D. Fox, "Imitating task and motion planning with visuomotor transformers," *arXiv preprint arXiv:2305.16309*, 2023.

[6] W. Shen, C. Garrett, N. Kumar, A. Goyal, T. Hermans, L. P. Kaelbling, T. Lozano-Pérez, and F. Ramos, "Differentiable gpu-parallelized task and motion planning," in *Robotics: Science and Systems*, 2025.

[7] L. Chen, S. R. Iyer, and Z. Kingston, "Differentiable particle optimization for fast sequential manipulation," *arXiv preprint arXiv:2510.07674*, 2025.

[8] C. Garrett and F. Ramos, "Schedulestream: Temporal planning with samplers for gpu-accelerated multi-arm task and motion planning & scheduling," *arXiv preprint arXiv:2511.04758*, 2025.

[9] D. A. Pomerleau, "Alvinn: An autonomous land vehicle in a neural network," in *Advances in neural information processing systems*, 1989, pp. 305–313.

[2]We disclose the use of Claude and ChatGPT to assist with code implementation, facilitating the experiments, and proofreading the manuscript.

[10] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, "What matters in learning from offline human demonstrations for robot manipulation," in *Conference on Robot Learning (CoRL)*, 2021.

[11] C. R. Garrett, A. Mandlekar, B. Wen, and D. Fox, "Skillgen: Automated demonstration generation for efficient skill learning and deployment," in *8th Annual Conference on Robot Learning*, 2024.

[12] L. Yang, H. Suh, T. Zhao, B. P. Graesdal, T. Kelestemur, J. Wang, T. Pang, and R. Tedrake, "Physics-driven data generation for contact-rich manipulation via trajectory optimization," *arXiv preprint arXiv:2502.20382*, 2025.

[13] C. Li, M. Xu, A. Bahety, H. Yin, Y. Jiang, H. Huang, J. Wong, S. Garlanka, C. Gokmen, R. Zhang *et al.*, "Momagen: Generating demonstrations under soft and hard constraints for multi-step bimanual mobile manipulation," *arXiv preprint arXiv:2510.18316*, 2025.

[14] K. Lin, A. Mandlekar, C. R. Garrett, N. Chernyadev, Y. Fang, R. Ding, Y. Xie, J. Tran, L. Fan, and Y. Zhu, "Humanoidmimicgen: Data generation for loco-manipulation via whole-body planning," *arXiv preprint arXiv:2605.27724*, 2026.

[15] D. Hadfield-Menell, C. Lin, R. Chitnis, S. Russell, and P. Abbeel, "Sequential quadratic programming for task plan optimization," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2016, pp. 5040–5047.

[16] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, "PDDLStream: Integrating Symbolic Planners and Blackbox Samplers," in *ICAPS*, 2020.

[17] A. Mandlekar, C. Garrett, D. Xu, and D. Fox, "Human-in-the-loop task and motion planning for imitation learning," in *7th Annual Conference on Robot Learning*, 2023.

[18] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins, "PDDL: The Planning Domain Definition Language," Yale Center for Computational Vision and Control, Tech. Rep., 1998.

[19] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett, K. Van Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, N. Ratliff, and D. Fox, "Curobo: Parallelized collision-free robot motion generation," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 8112–8119.

[20] N. Kumar, F. Ramos, D. Fox, and C. R. Garrett, "Open-world task and motion planning via vision-language model inferred constraints," in *Workshop on Language and Robot Learning: Language as an Interface*, 2024.

[21] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The Int'l Journal of Robotics Research*, 2023.

[22] M. Mittal, P. Roth, J. Tigue, A. Richard, O. Zhang, P. Du, A. Serrano-Munoz, X. Yao, R. Zurbrügg, N. Rudin *et al.*, "Isaac lab: A gpu-accelerated simulation framework for multi-modal robot learning," *arXiv preprint arXiv:2511.04831*, 2025.

[23] B. Sundaralingam, A. Murali, and S. Birchfield, "curobov2: Dynamics-aware motion generation with depth-fused distance fields for high-dof robots," *arXiv preprint arXiv:2603.05493*, 2026.

[24] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.

[25] R. Geraerts and M. H. Overmars, "Creating high-quality paths for motion planning," *The international journal of robotics research*, vol. 26, no. 8, pp. 845–863, 2007.

[26] X. Yang, R. Dagli, A. Zook, H. Hadfield, A. Goyal, S. Birchfield, F. Ramos, and J. Tremblay, "Robolab: A high-fidelity simulation benchmark for analysis of task generalist policies," *arXiv preprint arXiv:2604.09860*, 2026.

[27] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis *et al.*, "Droid: A large-scale in-the-wild robot manipulation dataset," *arXiv preprint arXiv:2403.12945*, 2024.

[28] A. Curtis, X. Fang, L. P. Kaelbling, T. Lozano-Pérez, and C. R. Garrett, "Long-horizon manipulation of unknown objects via task and motion planning with estimated affordances," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2022.

[29] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris Coll-Vinent, C. Ryali, K. V. Alwala, H. Khedr, A. Huang *et al.*, "Sam 3: Segment anything with concepts," in *International conference on learning representations*, vol. 2026, 2026, pp. 138 846–138 923.